Whitepaper: SkullCard's Zero-Knowledge Proof

Version 1.0
April 21st, 2026
Mourning Dove

Players in online poker have no way to verify the integrity of the deck being used against them. To prevent cheating, casinos cannot reveal the deck order or the cards of other players. A zero-knowledge proof allows players to confirm important properties of the deck without revealing card identities or deck order. These proofs reduce the trust players must place in the server distributing the cards. This whitepaper explains the circuit constraints, proof generation, and client-side verification incorporated into the SkullCard poker application.

Disclaimer: This document is not peer-reviewed academic work. It is an explanation of the zero-knowledge proof used by the SkullCard application, written to the best of the author's understanding. Errors are possible, if not plausible. The author welcomes feedback, suggestions, and corrections submitted as GitHub Issues at github.com/mourningdove007/skullcard.

Contents

1 Introduction **1**

2 Circuit **3**
2.1 Insufficient Uniqueness Constraints . . . 3
2.2 Matrix Solution . . . 4
2.3 Merkle Tree . . . 5

3 Prover **6**
3.1 SNARK Prover . . . 7
3.2 Merkle Path Calculation . . . 8

4 Verification **8**
4.1 Proof Verification . . . 8
4.2 Card Path Verification . . . 9

5 Future Work **10**
5.1 Commit-Reveal Protocol for Verifiable Randomness . . . 11

A Code **12**
A.1 Proof Verification Example . . . 12
A.2 snarkjs Verification . . . 13

1 Introduction

SkullCard [2] is an online poker platform built around a zero-knowledge proof protocol. The prover of the protocol is the server and the client is able to validate the proof. This is a reversal from the more common setup where the client is attempting to prove knowledge of a secret to the server. A

poker application seems like a natural fit for a server providing proof the deck used is the expected one. To prevent cheating, players should not be able to determine the cards of other players until during the round. The remaining cards in the deck after dealing should also be unknown to each player.

Claim 1. *The server will be able to prove to each client two statements without revealing player cards or deck order.*

1. *The deck the server is using contains a permutation of the unique 52 cards.*
2. *Client's cards were distributed from the same deck.*

Claim 2. *Requiring only client communication after receiving the proof from the server, one additional statement can be proven without revealing player cards or deck order.*

1. *Opponents have unique cards that were distributed from the same deck*

There are some important limitations to our current protocol. A user cannot be provided proof the deck is randomly shuffled. This allows a server to distribute cards in a biased manner. Our server makes use of salts to prevent brute force attacks that could reveal player cards. In the current implementation, the server generates the salts S and clients must trust they were drawn uniformly at random from $\mathbb{F}_p$. A malicious server could choose salts adversarially, for example reusing salts from a previous game to replay an old proof. This trust assumption is addressed in Section **??** where we describe a future protocol allowing clients to contribute their own randomness or the adoption of a blockchain solution.

The game can be started when two or more players, denoted P_i, are in a lobby. We assume that $n = 52$ in our discussion since the application is for a deck of cards. Cards are represented as integers c where $0 \leq c \leq p - 1$ where p is the prime order of the field $\mathbb{F}_p$. Our server shuffles the list $\{0, 1, \ldots, n-1\}$ to form a permutation $A = \{a_0, a_1, \ldots, a_{n-1} | a_j \in \mathbb{F}_p\}$. Salts are generated uniformly at random $S = \{s_0, s_1, \ldots, s_{n-1} | s_j \in \mathbb{F}_p\}$ and paired the permutation A forming $D = \{(a_0, s_0), (a_1, s_1), \ldots, (a_{n-1}, s_{n-1})\}$.[1] The server generates the proof π from D as discussed in Section 3.1. This proof π contains all information to provide proof of Claim 1. Hands $\mathcal{H}_i \subseteq D$ are selected and some special information $\mathcal{P}_i$ linking the hands to π are paired with each $\mathcal{H}_i$. We discuss $\mathcal{P}_i$ in detail in Section 2.3. The server sends $(\mathcal{H}_i, \mathcal{P}_i, \pi)$ to each P_i.

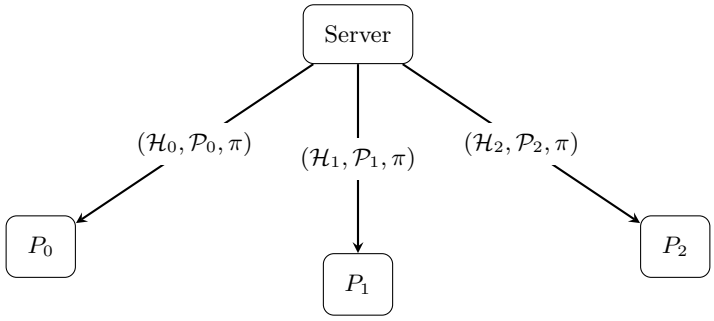

Figure 1: Server Distributes Hands and Proofs

Each P_i receives $(\mathcal{H}_i, \mathcal{P}_i, \pi)$ from the server. Using public information and $\mathcal{P}_i$, the client browser verifies the proof π. Players can then manually share $\mathcal{P}_i$ with their opponents to establish Claim 2.

[1] Shuffle and salt generation are implemented by the Rust Shuffle API: https://github.com/mourningdove007/skullcard/tree/main/zk/rust.

Figure 2 shows what is exchanged between each player, P_i. $\mathcal{P}_i$ hides information about the cards but confirms they are unique from other players' cards and from the same deck. The only value P_i must hide is $\mathcal{H}_i$ which reveals their hand. After the round is complete, A and S are dropped, and the process resets.

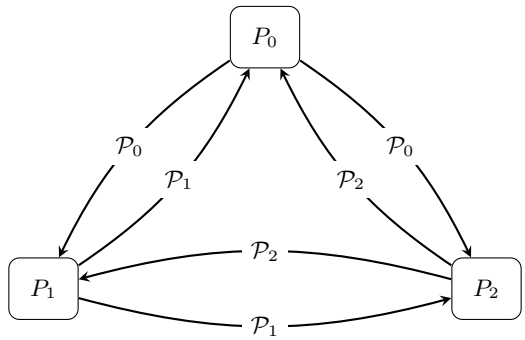

Figure 2: Player to Player Broadcast

2 Circuit

Given $A = \{a_0, a_1, \ldots, a_{n-1} | a_i \in \mathbb{F}_p\}$, and some salts $S = \{s_0, s_1, \ldots, s_{n-1} | s_i \in \mathbb{F}_p\}$, we want our circuit to be able to do two tasks:

1. Prove that A is a permutation of $\{0, 1, 2, \ldots, n-1\}$.
2. Output a Merkle root provably derived from A and S.

Compiling the circuit with Circom produces a proving key pk and a verification key vk, both available open source at [4]. These keys are used by the proving and verification processes described in the next sections.

2.1 Insufficient Uniqueness Constraints

A permutation of $\{0, 1, 2, \ldots, n-1\}$ has a couple known properties that could be used to establish simple constraints. Define the sum and sum-of-squares for a set $A = \{a_0, a_1, \ldots, a_{n-1}\}$ as follows:

$$\sum_{i=0}^{n-1} a_i \qquad \sum_{i=0}^{n-1} a_i^2$$

The following claim can be proven through induction.

Claim 3. *Sum and Sum-of-Squares for* $\{0,\ 1,\ \ldots,\ n-1\}$

$$\sum_{i=0}^{n-1} i = \frac{n(n+1)}{2} \qquad \sum_{i=0}^{n-1} i^2 = \frac{n(n+1)(2n+1)}{6}$$

An efficient constraint can be constructed by calculating the sum and sum-of-squares of a given list. A constraint on the sum and sum-of-squares is, however, not sufficient for proving the deck is a permutation of $\{0, 1, 2, \ldots, n-1\}$.

Claim 4. *Let $A = \{a_0, a_1, \ldots, a_{n-1}\}$ be a list of non-negative integers where each element is strictly less than n and*

$$\sum_{i=0}^{n-1} a_i = \frac{n(n-1)}{2} \quad \textit{and} \quad \sum_{i=0}^{n-1} a_i^2 = \frac{n(n-1)(2n-1)}{6}$$

There exists A which is not a permutation of $\{0, 1, \ldots, n-1\}$ when $n > 44$.

Proof. Swap out the values 22, 27, 44 for 20, 30, 43 and we have a different solution for any $n > 44$.

$$22 + 27 + 44 = 20 + 30 + 43 = 93$$
$$22^2 + 27^2 + 44^2 = 20^2 + 30^2 + 43^2 = 3149$$

This proves the sum-of-squares and sum constraints are not enough to guarantee uniqueness. □

2.2 Matrix Solution

Sum-of-squares and the sum provided a constraint that was resource efficient but insufficient for our requirements. An alternative solution is required to correctly prove uniqueness. Consider an $n \times n$ binary matrix $\mathcal{M}$. The entry in $\mathcal{M}$ at row v column i is denoted $\mathcal{M}_{v,i}$. Since $\mathcal{M}$ is binary, $\mathcal{M}_{v,i} \in \{0, 1\}$ for all $v, i \in \{0, 1, 2, \ldots n-1\}$. A is a set of n elements $\{a_0, a_1, \ldots, a_{n-1}\}$ that are not necessarily unique. Define $\mathcal{M}(A)$ by

$$\mathcal{M}(A) = \begin{bmatrix} \mathcal{M}_{0,0} & \mathcal{M}_{0,1} & \ldots & \mathcal{M}_{0,n-1} \\ \mathcal{M}_{1,0} & \mathcal{M}_{1,1} & \ldots & \mathcal{M}_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathcal{M}_{n-1,0} & \mathcal{M}_{n-1,1} & \ldots & \mathcal{M}_{n-1,n-1} \end{bmatrix} \qquad \mathcal{M}_{v,i} = \begin{cases} 1 & \text{if } a_i = v \\ 0 & \text{if } a_i \neq v \end{cases}$$

Each element in A has to be unique if A is a permutation of {0, 1, . . . , n-1}. The expectation is only one nonzero entry in each row v should exist. That entry will exist only when $a_i = v$ by definition. Example 3 gives evidence of only one value in each row for a permutation of {0, 1, 2, 3}.

$$\begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \qquad \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Figure 3: Binary matrices for $\mathcal{M}([1, 3, 0, 2])$ (left) and $\mathcal{M}([1, 1, 2, 2])$ (right).

A constraint is set in the circuit that checks the sum of each row.

Constraint 1. *Let $A = \{a_0, a_1, \ldots, a_{n-1}\}$. The $\mathcal{M}(A)$ must satisfy the equality*

$$\sum_{i=0}^{n-1} \mathcal{M}_{v,i} = 1$$

for all $v \in \{0, 1, \ldots, n-1\}$. The constraint holds if and only if A is a permutation of $\{0, 1, \ldots, n-1\}$.

Proof. In the reverse direction, suppose A is a permutation of $\{0, 1, \ldots, n-1\}$. Consider $\{\mathcal{M}_{v,i}\}_{i=0}^{n-1}$ which is a row of entries in $\mathcal{M}$ for some $v \in \{0, 1, \ldots, n-1\}$. A is a permutation of $\{0, 1, \ldots, n-1\}$ so there is exactly one v at an index k in A. Only $\mathcal{M}_{v,k} \in \{\mathcal{M}_{v,i}\}_{i=0}^{n-1}$ equals 1 and the sum is 1. This same argument is used for all rows since v is generalized so $\sum_{i=0}^{n-1} \mathcal{M}_{v,i} = 1$ holds.

Now suppose $\sum_{i=0}^{n-1} \mathcal{M}_{v,i} = 1$ for all $v \in \{0, 1, \ldots, n-1\}$. Consider again a single row for a fixed value v: $\{\mathcal{M}_{v,i}\}_{i=0}^{n-1}$. $\mathcal{M}(A)$ is binary so entries can only be 0 or 1. Having no nonzero entries leads to a sum of zero. Having more than one nonzero entry leads to a sum of at least two. Each is contradicting the initial claim so there must be a single nonzero value $\mathcal{M}_{v,k} \in \{\mathcal{M}_{v,i}\}_{i=0}^{n-1}$. Then $v = a_k$ and $v \in A$. This holds true for all $v \in \{0, 1, \ldots, n-1\}$. A is assumed to only have n elements so those must be the same elements in $\{0, 1, \ldots, n-1\}$ □

```
pragma circom 2.0.0;

include "node_modules/circomlib/circuits/comparators.circom
    ";

template PermutationCheck(n) {
    signal input in[n];
    signal occurrences[n][n];

    for (var v = 0; v < n; v++) {
        var rowSum = 0;
        for (var i = 0; i < n; i++) {
            occurrences[v][i] <== IsEqual()([in[i], v]);

            rowSum += occurrences[v][i];
        }
        rowSum === 1;
    }
}

component main = PermutationCheck(52);
```

Figure 4: Circom [8] implementation of the permutation check for $n = 52$, generating $52^2 = 2{,}704$ `IsEqual` constraints.

2.3 Merkle Tree

After verifying the permutation constraint, the circuit commits to the deck by building a Merkle tree over the 52 cards. The root of this tree serves as a public output that the verifier can use to confirm that a card was drawn from the same deck the server committed to. Each card is hashed together with a salt using a hash function H.

A *hash function over the field* $\mathbb{F}_p$, $H : \mathbb{F}_p^* \to \mathbb{F}_p$, is a function satisfying the following properties:

- Given y, it is computationally infeasible to find x such that $H(x) = y$.
- Given x, it is computationally infeasible to find $x' \neq x$ such that $H(x') = H(x)$.
- It is computationally infeasible to find any pair $x \neq x'$ such that $H(x) = H(x')$.

Poseidon [6] is the hash function used in our circuit implementation. To keep the discussion generalized, we will use H to describe our hash function. This leaves us the option to adjust the hash function in future implementations.

A salt prevents another player from brute-forcing card values from a leaf hash. Let c_i, s_i be the ith entry of card and salt input, respectively. The leaf of the Merkle tree at position i, denoted ℓ_i, is calculated as

$$\ell_i = H(c_i, s_i)$$

We will assume our Merkle trees are binary trees with 2^k leaves. The circuit is instantiated with $n = 52$ cards and `nTotalLeaves` $= 64$ leaves, the smallest power of two greater than or equal to 52. Twelve padding leaves are filled with $H(0, 0)$ bringing the total to 64 leaves. Populating the Merkle tree branches begins by pairing the 64 leaf hashes into 32 parent nodes demonstrated in Figure 5. In the implementation, parent nodes are stored in the array `upperHashes`.

```
for (var i = 0; i < 32; i++) {
    upperHashes[i] = Poseidon(2);
    upperHashes[i].inputs[0] <==
        leafHashes[2*i].out;
    upperHashes[i].inputs[1] <==
        leafHashes[2*i+1].out;
}
```

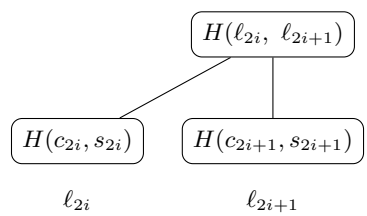

Figure 5: Pairing leaf hashes into parent nodes

Parent hashes are paired level by level until a single node remains. This process continues for 5 levels, terminating at the Merkle root. The root is the public output of the circuit, referred to as a *public signal*, and serves as a commitment to the deck. Players use this signal to verify their cards belong to the deck as described in Section 4. An argument by induction can be made to show a binary Merkle tree with N leaves has exactly $N - 1 = 63$ internal nodes. Indexes begin at zero so the last internal node in the tree would be at index $N - 2$ so the root occupies the final position `upperHashes[nTotalLeaves - 2]`.

3 Prover

Our server handles the following tasks:

1. Generate a permutation A of the set $\{0, 1, \ldots, n-1\}$ and some salts $S = \{s_0, s_1, \ldots, s_{n-1} | s_i \in \mathbb{F}_p\}$
2. Generate witness and signals from the compiled circuit, A, and S
3. Distribute the proof π calculated from the witness and signals
4. Distribute to each player their dealt cards along with the corresponding Merkle leaves and paths.

A SNARK prover uses the compiled circuit and a proving key to produce a proof. Server-side logic computes the Merkle path for each player's cards. The resulting proof and Merkle paths are transmitted to the client. All this information is produced without initial client interaction resulting in a non-interactive proof.

```
var offset = 0;
var next_offset = 32;
for (var levelSize = 16; levelSize >= 1; levelSize /= 2) {
    for (var i = 0; i < levelSize; i++) {
        upperHashes[next_offset + i] = Poseidon(2);
        upperHashes[next_offset + i].inputs[0] <==
            upperHashes[offset + 2*i].out;
        upperHashes[next_offset + i].inputs[1] <==
            upperHashes[offset + 2*i + 1].out;
    }
    offset = next_offset;
    next_offset += levelSize;
}

root <== upperHashes[nTotalLeaves - 2].out;
```

Figure 6: Populating the Merkle Tree

3.1 SNARK Prover

Our protocol requires use of a proof system called a SNARK (Succinct Non-interactive Argument of Knowledge). With the circuit compiled and pk in hand, the prover holds the private inputs A and S and uses them as a witness $w = (A, S)$ to produce a proof. Given pk and w, the SNARK prover computes:

$$\pi, x = \text{Prove}(pk,\ w)$$

producing a proof $\pi = (\pi_a, \pi_b, \pi_c)$ where $\pi_a, \pi_c \in \mathbb{G}_1$ and $\pi_b \in \mathbb{G}_2$. x are the public signals produced by the circuit. In our protocol, x is the Merkle root.

The proof guarantees that the prover knew a witness satisfying every arithmetic constraint of the circuit and that evaluating the circuit on that witness produces the committed Merkle root x. Only x and verification key vk are required by the client to verify the proof. There is an existing UI feature allowing the client to verify proofs in their browser but users can verify themselves by using the verification key. In Section 4, we demonstrate how the proof can be verified using the `py_ecc` library [5].

Our implementation uses the SNARK prover called Groth16 [7] implemented by the `snarkjs` library [10]. A full test suite of the implementation can be found in the project repository [3]. Using the publicly available proving key and WASM witness generator from the same repository, we create a proof with the following javascript:

```
const deck = Array.from({ length: 52 }, (_, i) => i);
const salts = Array(52).fill(12345n);

const input = { "cards": deck, "salts": salts };

const { proof, publicSignals } = await snarkjs.groth16.fullProve(
    input,
    "shuffle_js/shuffle.wasm",
    "shuffle_final.zkey"
);
```

3.2 Merkle Path Calculation

For each leaf ℓ_j in a player's hand, the server computes the corresponding path $Path_{i,j} \in \mathcal{P}_i$, consisting of the sibling hash L_k and direction d_k at each level of the tree. A player can reconstruct $\ell_j = H(c_j, s_j)$ from their own card index and salt, so the path need only supply the (L_k, d_k) pairs required to recompute the root x.

```
function getPath(tree, leafIndex) {
  const path = [];
  let index = leafIndex;

  for (let level = 0; level < tree.levels.length - 1; level
    ++) {
    const isRight = index % 2 === 1;
    const siblingIndex = isRight ? index - 1 : index + 1;
    path.push({
      sibling: tree.levels[level][siblingIndex],
      direction: isRight ? 1 : 0,
    });
    index = Math.floor(index / 2);
  }

  return path;
}
```

Figure 7: Server-side computation of $Path_{i,j}$: at each level the sibling node and direction $d_k \in \{0, 1\}$ are collected, matching the convention used in $\mathcal{P}_i$.

4 Verification

The user interface of SkullCard handles the verification process in the client-browser with the snarkjs and circomlib [9] libraries. Our implementation to verify the server proof is found in the Appendix A.2. A full test suite of the verification process can be found in the project repository [3]. Users are able to manually do these calculations and are not required to use the user interface solution. To demonstrate the verification process, the following sections will manually verify the proofs without the use of the JavaScript libraries.

4.1 Proof Verification

Each player receives the proof $\pi = (\pi_a, \pi_b, \pi_c)$ and public signal x (the Merkle root) from the server. Verification requires a publicly shared verification key vk, available at [1]. With x, π, and vk, a player can also check the proof manually using the ecc_py library [5].

```
from py_ecc.bn128 import G1, G2, pairing, multiply, add, neg, FQ, FQ2
import json
```

A player can compute a group element $vk_x \in \mathbb{G}_1$ from the public signal and the verification key. The verification key is of the form $vk = (\alpha, \beta, \gamma, \delta, \{u_0, u_1\})$ where u_i are referred to as input commitments.

```
vk = json.load(open("verification_key.json"))
```

```
alpha = parse_g1(vk["vk_alpha_1"])
beta  = parse_g2(vk["vk_beta_2"])
gamma = parse_g2(vk["vk_gamma_2"])
delta = parse_g2(vk["vk_delta_2"])
IC    = [parse_g1(p) for p in vk["IC"]]

vk_x = IC[0]
for i, signal in enumerate(public_signals):
    vk_x = add(vk_x, multiply(IC[i + 1], int(signal)))
```

Let $\mathbb{G}_T$ be a multiplicative subgroup of $\mathbb{F}^*_{p^{12}}$, required by the Groth16 protocol [7]. A mapping $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is *bilinear* if $e(aP, bQ) = e(P, Q)^{ab}$ for all $P \in \mathbb{G}_1$, $Q \in \mathbb{G}_2$, and $a, b \in \mathbb{Z}$. The Python library ecc_py has the required bilinear function we need for the groth16 proof called `pairing`. We check for the following equality to hold to verify the proof:

$$e(\pi_a, \pi_b) = e(\alpha, \beta) \cdot e(vk_x, \gamma) \cdot e(\pi_c, \delta)$$

```
lhs = pairing(pi_b, pi_a)

rhs = (pairing(beta,  alpha) *
       pairing(gamma, vk_x)  *
       pairing(delta, pi_c))

print("Proof valid:", lhs == rhs)
```

The full demo verification process can be found in the Appendix A.1.

4.2 Card Path Verification

The Merkle root x can be verified by the card paths $\mathcal{P}_i = \{Path_{i,0}, Path_{i,1}, \ldots, Path_{i,4}\}$ and hand $\mathcal{H}_i$ for player P_i. Hands are limited to 5 cards so $|\mathcal{H}_i| = 5$. Paths contain the parent hash L_k and direction $d_k \in \{0, 1\}$ at each level of the tree for a particular Merkle leaf, ℓ_j. Players can calculate the Merkle leaf from their hand $\ell_j = H(c_j, s_j)$ where $(c_j, s_j) \in \mathcal{H}_i$. The path for a card in $\mathcal{H}_i$ can be defined by $Path_{i,j} = \{(L_0, d_0), (L_1, d_1), \ldots (L_5, d_5)\}$

The direction determines which side the input occurs when applying the hash H. If $d_k = 0$, then we will use $H_0(input, L_k)$ and we will use $H_1(input, L_k)$ when $d_k = 1$.

$$H_0(input, L_k) = H(input, L_k)$$

$$H_1(input, L_k) = H(L_k, input)$$

To verify the card is from the committed deck, calculate h_5 and verify that h_5 is equal to the Merkle root.

$$\begin{aligned} h_0 &= H_{d_0}(\ell_j, L_0) \\ h_1 &= H_{d_1}(h_0, L_1) \\ &\vdots \\ h_5 &= H_{d_5}(h_4, L_4) \end{aligned}$$

Table 1 gives an example path $\mathcal{P}_0$ returned by the server. Each level is hashed using the Poseidon library as follows:

```
const { buildPoseidon } = require('circomlibjs');
const poseidon = await buildPoseidon();
const F = poseidon.F;

hash = F.toString(poseidon([BigInt(L_0), BigInt(l)]))
let current = BigInt(hash)
```

Applying this at each level produces the intermediate hashes in Table 2, with the final hash equal to the Merkle root.

ℓ
10872828716592724965192164088009820922770757764273804244211728952469366442950
Merkle Root
13222794172581909933692251382405521232784303702471687528735583494052096354 62
d_0
1
L_0
19034655021735546934465701229526099046563740865544468522735582910355488045441
d_1
0
L_1
84442852674915161972879058645324353998376300408204080670257856935655286145 91
d_2
1
L_2
13891839695747085780472302533923138205991821012633183092240429179808274252972
d_3
0
L_3
44653522064888846828897123572767658130141848316123090118003795724789768255 45
d_4
0
L_4
21171445598686220504438449214462587155614312041751227177597072181794203039 02
d_5
0
L_5
91659470359427751849202881683819864558045776391561441376870174837176331538 72

Table 1: Path data for $\mathcal{P}_0$: leaf ℓ, Merkle root x, and sibling hashes L_k with directions d_k at each level.

Currently, the UI only verifies the Merkle leaves of the player and does not support exchanging paths. We plan to support this feature in a future version of the app. Players will be able to manually exchange their leaves to verify. Each player can share $\mathcal{P}_i$ with the other players in the game. Paths only need to contain the Merkle leaf ℓ_j and not the card indexes and salts. As long as each path in $\mathcal{P}_i$ leads to the calculation of the Merkle root, players can be assured the cards came from the committed deck.

5 Future Work

The current protocol leaves one area where a trust assumption could be eliminated with additional engineering. This section outlines the planned improvement.

$H_1(\ell, L_0)$ 999034248901148565896075366874384739970953208269105845557546882530228007 0044
$H_0(h_0, L_1)$ 65638694760512837176496984378114918263237132445226785891321144379807895583 74
$H_1(h_1, L_2)$ 15069907103078539571433278313183272187278297799151889914754312759665068087742
$H_0(h_2, L_3)$ 16905911700132723373337913877781780131055975283070594479641777130743070809393
$H_0(h_3, L_4)$ 21609006617036325792470889326837059895239049598017190043152817571479060755531
$H_0(h_4, L_5)$ 13222794172581909933692251382405521232784303702471687528735583494052096354 62

Table 2: Intermediate hashes computing the Merkle root from ℓ along $\mathcal{P}_0$. The final value equals x.

5.1 Commit-Reveal Protocol for Verifiable Randomness

Salt generation, deck shuffling, and Merkle leaf construction are currently handled by a dedicated Rust application operating as an external server. Rust was chosen for its memory safety guarantees and performance characteristics; the isolation of these operations into a single auditable service reduces the attack surface compared to embedding them in the Firebase Cloud Functions layer. However, clients must still trust that the Rust server draws salts and shuffle order uniformly at random, and does not reuse randomness from prior games or choose values adversarially to bias outputs.

The planned improvement is a commit-reveal protocol that removes this trust assumption entirely by making all randomness a collaborative product of the players themselves. Before the game begins, each of the m players independently generates a random contribution $v_j \in \mathbb{F}_p$, computes a commitment $c_j = H(v_j)$, and broadcasts c_j to all participants. Once all m commitments have been shared with all participants, players send v_j to the server. Players are required to commit to a value to prevent collusion between a dishonest server and a dishonest player. If the server were to reveal the other players' randomness to a dishonest player before that player committed, the dishonest player could calculate their own value in a biased manner.

The master seed is derived as:

$$\sigma = H(c_0 \| c_1 \| \cdots \| c_{m-1})$$

and used as the input to a deterministic pseudorandom function, which drives both the salt vector $S = \{s_0, s_1, \ldots, s_{n-1}\}$ and the deck shuffle permutation π used to construct the Merkle tree. The values v_j are kept secret by the server for the duration of the game. Revealing them during play would allow any party to recompute the output of the deterministic pseudorandom function, enabling a dishonest user to derive the full deck ordering. After the game concludes, the server posts the randomness contributions v_j. Any participant can then verify that $H(v_j) = c_j$ for each j, confirm that σ was honestly derived from the committed values, and re-run the PRF to audit the salt vector and deck ordering. As long as at least one player generated their contribution honestly, σ is computationally indistinguishable from uniform.

A Code

A.1 Proof Verification Example

```
from py_ecc.bn128 import pairing, multiply, add, FQ, FQ2
import json

def parse_g1(point):
    return (FQ(int(point[0])), FQ(int(point[1])))
def parse_g2(point):
    return (
        FQ2([int(point[0][0]), int(point[0][1])]),
        FQ2([int(point[1][0]), int(point[1][1])]),
    )

pi_a = parse_g1(pi_a_array)
pi_b = parse_g2(pi_b_array)
pi_c = parse_g1(pi_c_array)
public_signals = public_signals_array
vk = json.load(open("verification_key.json"))
alpha = parse_g1(vk["vk_alpha_1"])
beta  = parse_g2(vk["vk_beta_2"])
gamma = parse_g2(vk["vk_gamma_2"])
delta = parse_g2(vk["vk_delta_2"])
IC    = [parse_g1(p) for p in vk["IC"]]

vk_x = IC[0]
for i, signal in enumerate(public_signals):
    vk_x = add(vk_x, multiply(IC[i + 1], int(signal)))

lhs = pairing(pi_b, pi_a)
rhs = (pairing(beta,  alpha) *
       pairing(gamma, vk_x)  *
       pairing(delta, pi_c))

print("Proof valid:", lhs == rhs)
```

A.2 snarkjs Verification

```
import { assert } from "chai";
import * as snarkjs from "snarkjs";
import fs from "fs";

describe("Shuffle Verification Test", function () {
    this.timeout(100000); // Proving takes a few seconds

    it("Should generate and verify a shuffle proof", async
    () => {
        const deck = Array.from({ length: 52 }, (_, i) => i)
    ;
        const salts = Array(52).fill(12345n);

        const input = { "cards": deck, "salts": salts };

        const { proof, publicSignals } = await snarkjs.
    groth16.fullProve(
            input,
            "shuffle_js/shuffle.wasm",
            "shuffle_final.zkey"
        );

        // Merkle Root
        const root = publicSignals[0];

        // Verify
        const vKey = JSON.parse(fs.readFileSync("
    verification_key.json"));
        const res = await snarkjs.groth16.verify(vKey,
    publicSignals, proof);

        assert.isTrue(res, "The proof should be valid");
    });


});
```

References

[1] Mourning Dove. Skullcard: Groth16 verification key. https://github.com/mourningdave007/skullcard/blob/main/zk/circuits/verification_key.json, 2026.

[2] Mourning Dove. Skullcard: Zero-knowledge poker. https://skullcard.com, 2026.

[3] Mourning Dove. Skullcard: ZK circuit implementation and tests. https://github.com/mourningdave007/skullcard/tree/main/zk/circuits, 2026.

[4] Mourning Dove. Source code for skullcard. https://github.com/mourningdove007/skullcard, 2026.

[5] Ethereum Foundation. py_ecc: Elliptic curve cryptography in python. https://github.com/ethereum/py_ecc, 2023.

[6] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. POSEIDON: A new hash function for zero-knowledge proof systems. Cryptology ePrint Archive, Report 2019/458, 2019.

[7] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326. Springer, 2016.

[8] Iden3. Circom: A circuit description language for building zero-knowledge applications. https://docs.circom.io, 2023.

[9] Iden3. circomlib: Library of circuit templates for circom. https://github.com/iden3/circomlib, 2023.

[10] Iden3. snarkjs: zkSNARK implementation in javascript and WASM. https://github.com/iden3/snarkjs, 2023.